```
import numpy as np
import tensorflow as tf
from tensorflow import keras
```

Datasset Preparation

```
np.random.seed(42)

X = np.random.rand(1000, 1)
y = 3 * X + 2 + np.random. randn(1000, 1) * 0.1
```

Model definition

```
def build_model():
    model = tf.keras.Sequential([
        tf.keras.layers.Dense(1, input_shape=[1])
    ])
    return model

loss_func = tf.keras.losses.MeanSquaredError()
```

Experiment 1: Batch Gradient Descent

```
model_batch_sgd = build_model()
model_batch_sgd.compile(optimizer='sgd', loss='mean_squared_error')
history_batch_sgd = model_batch_sgd.fit(X, y, batch_size=len(X), verbose=1,

print(f"Final loss: {history_batch_sgd.history['loss'][-1]:.4f}")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 140ms/step - loss: 0.3870
Epoch 82/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 199ms/step - loss: 0.3758
Epoch 83/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 190ms/step - loss: 0.3652
Epoch 84/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 67ms/step - loss: 0.3551
Epoch 85/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 64ms/step - loss: 0.3454
Epoch 86/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 73ms/step - loss: 0.3362
Epoch 87/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 72ms/step - loss: 0.3275
Epoch 88/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 51ms/step - loss: 0.3191
Epoch 89/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step - loss: 0.3112
Epoch 90/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step - loss: 0.3036
Epoch 91/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 42ms/step - loss: 0.2964
Epoch 92/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.2895
Epoch 93/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step - loss: 0.2829
Epoch 94/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step - loss: 0.2767
Epoch 95/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 42ms/step - loss: 0.2707
Epoch 96/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step - loss: 0.2650
Epoch 97/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 49ms/step - loss: 0.2596
Epoch 98/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step - loss: 0.2545
Epoch 99/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step - loss: 0.2495
Epoch 100/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 54ms/step - loss: 0.2448
Final loss: 0.2448
```

Experiment 2: Stochastic Gradient Descent

```
model_sgd = build_model()
model_sgd.compile(optimizer='sgd', loss=loss_func)
history_sgd = model_sgd.fit(X, y, batch_size=1, verbose=1, epochs = 30)

print(f"Final loss: {history_sgd.history['loss'][-1]:.4f}")
```

```
Epoch 1/30
1000/1000 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - loss: 1.1084
Epoch 2/30
1000/1000 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - loss: 0.0139
Epoch 3/30
1000/1000 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - loss: 0.0111
Epoch 4/30
```

```
1000/1000 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - loss: 0.0099
Epoch 5/30
1000/1000 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - loss: 0.0100
Epoch 6/30
1000/1000 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - loss: 0.0101
Epoch 7/30
1000/1000 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - loss: 0.0095
Epoch 8/30
1000/1000 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - loss: 0.0105
Epoch 9/30
1000/1000 ━━━━━━━━━━━━━━━━━━━━ 2s 1ms/step - loss: 0.0101
Epoch 10/30
1000/1000 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - loss: 0.0102
Epoch 11/30
1000/1000 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - loss: 0.0100
Epoch 12/30
1000/1000 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - loss: 0.0098
Epoch 13/30
1000/1000 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - loss: 0.0092
Epoch 14/30
1000/1000 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - loss: 0.0104
Epoch 15/30
1000/1000 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - loss: 0.0094
Epoch 16/30
1000/1000 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - loss: 0.0093
Epoch 17/30
1000/1000 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - loss: 0.0100
Epoch 18/30
1000/1000 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - loss: 0.0096
Epoch 19/30
1000/1000 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - loss: 0.0098
Epoch 20/30
1000/1000 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - loss: 0.0101
Epoch 21/30
1000/1000 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - loss: 0.0101
Epoch 22/30
1000/1000 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - loss: 0.0104
Epoch 23/30
1000/1000 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - loss: 0.0105
Epoch 24/30
1000/1000 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - loss: 0.0102
Epoch 25/30
1000/1000 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - loss: 0.0103
Epoch 26/30
1000/1000 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - loss: 0.0104
Epoch 27/30
1000/1000 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - loss: 0.0095
Epoch 28/30
1000/1000 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - loss: 0.0101
Epoch 29/30
```

Experiment 3: Mini-batch grqdient descent

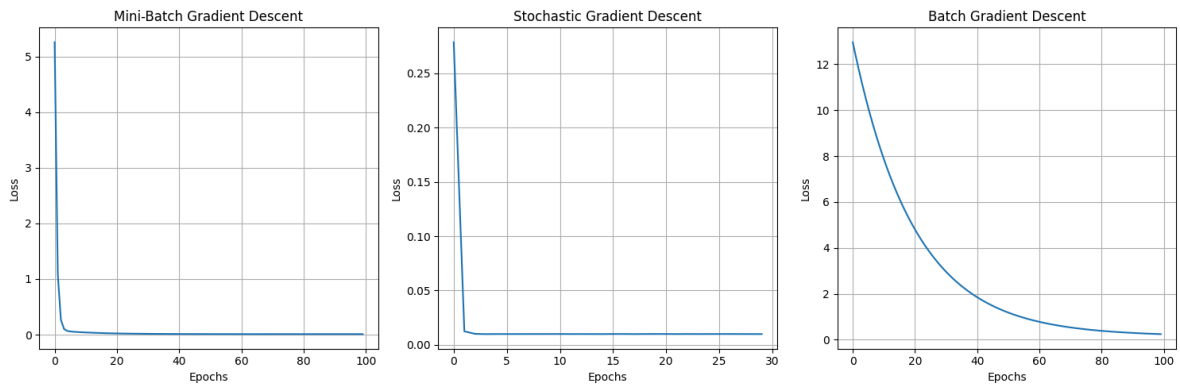
```
model_mini_batch = build_model()
model_mini_batch.compile(optimizer='sgd', loss=loss_func)

history_mini_batch = model_mini_batch.fit(X, y, batch_size=32, epochs=100, v
```

```
print(f"Final loss: {history_mini_batch.history['loss'][-1]:.4f}")
```

```
Epoch 28/100
```

```
32/32 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 0.0166
```

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(1, 3, figsize=(15, 5))

ax[0].plot(history_mini_batch.history['loss'])
ax[0].set_title("Mini-Batch Gradient Descent")
ax[0].set_xlabel("Epochs")
ax[0].set_ylabel("Loss")
ax[0].grid(True)

ax[1].plot(history_sgd.history['loss'])
ax[1].set_title("Stochastic Gradient Descent")
ax[1].set_xlabel("Epochs")
ax[1].set_ylabel("Loss")
ax[1].grid(True)

ax[2].plot(history_batch_sgd.history['loss'])
ax[2].set_title("Batch Gradient Descent")
ax[2].set_xlabel("Epochs")
ax[2].set_ylabel("Loss")
ax[2].grid(True)

plt.tight_layout()
plt.show()
```



Answers to Exercise 1 Questions

1. **Why does Batch Gradient Descent have a smooth loss curve but slow convergence?**
    - **Smooth Loss Curve:** Batch Gradient Descent (BGD) computes the gradient using the entire training dataset for each update. This provides a

very accurate and stable estimate of the true gradient, leading to a smooth and consistent decrease in the loss function over epochs.
    - **Slow Convergence:** Because it processes the entire dataset before each parameter update, BGD can be very slow, especially for large datasets. Each iteration is computationally expensive, even if it takes fewer overall iterations to reach the minimum compared to other methods.

2. **What causes fluctuations in the loss function when using SGD?**
    - Stochastic Gradient Descent (SGD) updates parameters after processing *each single training example*. The gradient calculated from a single example can be noisy and might not accurately represent the true gradient of the entire loss function. This noise causes the loss function to fluctuate and oscillate around the minimum, rather than converging smoothly.
3. **Why is Mini-Batch Gradient Descent preferred for training deep neural networks?**
    - Offers a good balance between the stability of BGD and the speed of SGD.
    - Computationally much faster than BGD, while still providing more stable gradient estimates than SGD.
    - Modern computing hardware (like GPUs) is highly optimized for vectorized operations, making mini-batch computations very efficient.
    - Provides a smoother convergence path than SGD, while being much faster than BGD.
4. **What happens if the batch size is increased to the full dataset?**
    - If the batch size is increased to the full dataset, Mini-Batch Gradient Descent effectively becomes **Batch Gradient Descent (BGD)**. This means each parameter update will be based on the gradient computed from all training examples. As discussed, this leads to a very smooth loss curve but is computationally expensive and slow per update, especially for large datasets.
5. **How does shuffling the dataset affect SGD and Mini-Batch GD?**
    - **Prevents Bias and Cycles:** Without shuffling, if the dataset has an inherent order (e.g., all examples of one class followed by all examples of another), the model might learn biased patterns or get stuck in cycles, optimizing for one part of the data at the expense of another.
    - **Ensures Diverse Gradients:** Shuffling ensures that each mini-batch or single example in SGD is a diverse and more representative sample of the entire dataset. This leads to more accurate and less biased gradient estimates over time, helping the model converge more effectively to a good solution and generalize better.

Exercise 1: Effect of Learning Rate

Objective: To study the effect of learning rate on convergence behavior.
Task: - Train the same model using Mini-Batch Gradient Descent - Use learning rates: 0.001, 0.01, and 0.1 - Plot loss vs epochs for each case

```
model_lr_01 = build_model()
model_lr_01.compile(optimizer=tf.keras.optimizers.SGD(learning_rate=0.1), lo
history_lr_01 = model_lr_01.fit(X, y, batch_size=32, epochs=100, verbose=0)

print(f"Training complete for learning rate 0.1. Final loss: {history_lr_01.
```

```
Training complete for learning rate 0.1. Final loss: 0.0098
```

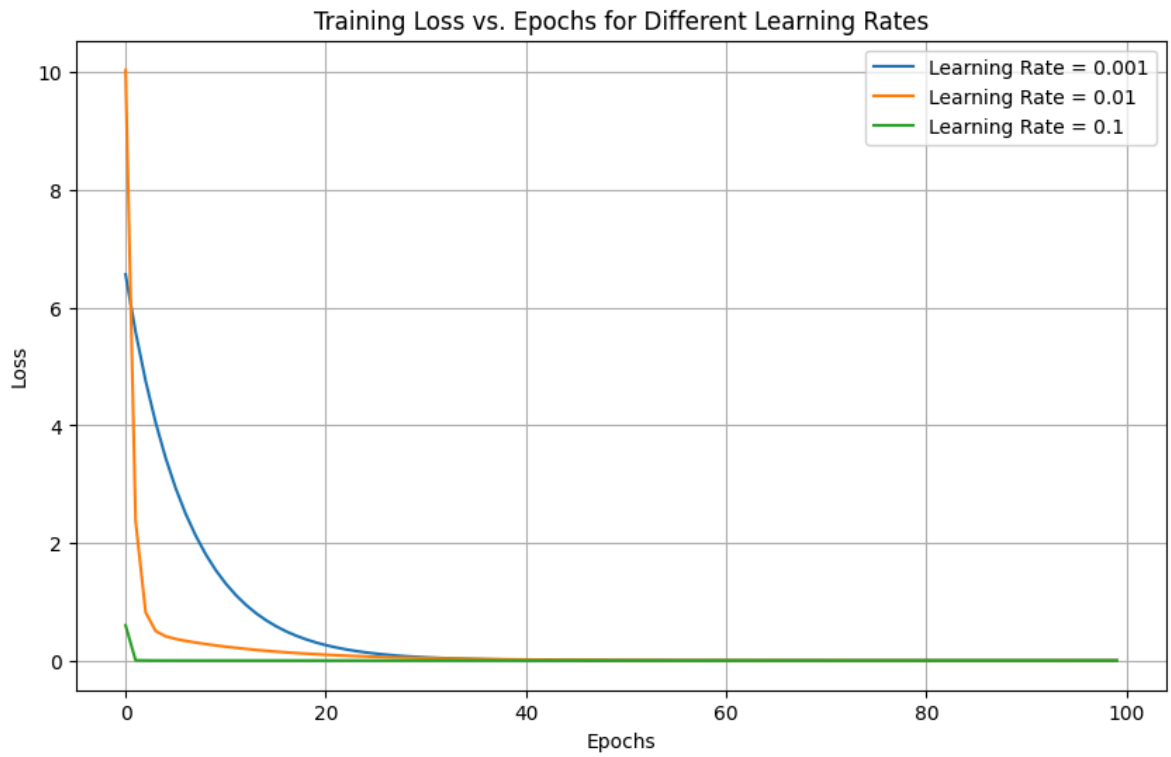
```
model_lr_001 = build_model()
model_lr_001.compile(optimizer=tf.keras.optimizers.SGD(learning_rate=0.01),
history_lr_001 = model_lr_001.fit(X, y, batch_size=32, epochs=100, verbose=0

print(f"Training complete for learning rate 0.01. Final loss: {history_lr_00
```

```
Training complete for learning rate 0.01. Final loss: 0.0099
```

```
model_lr_0001 = build_model()
model_lr_0001.compile(optimizer=tf.keras.optimizers.SGD(learning_rate=0.001)
history_lr_0001 = model_lr_0001.fit(X, y, batch_size=32, epochs=100, verbose

print(f"Training complete for learning rate 0.001. Final loss: {history_lr_0
```

```
Training complete for learning rate 0.001. Final loss: 0.0106
```

```
plt.figure(figsize=(10, 6))
plt.plot(history_lr_0001.history['loss'], label='Learning Rate = 0.001')
plt.plot(history_lr_001.history['loss'], label='Learning Rate = 0.01')
plt.plot(history_lr_01.history['loss'], label='Learning Rate = 0.1')
plt.title('Training Loss vs. Epochs for Different Learning Rates')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
plt.show()
```



Answers to Exercise 1 Questions

1. **Which learning rate converges fastest?**
   - Based on the plot, the learning rate of **0.1** converges the fastest, showing the steepest drop in loss during the initial epochs and reaching a low loss value quickly.
2. **Which learning rate causes instability?**
   - Learning rate **0.1** shows slight instability, with a more jagged initial descent compared to 0.01. While it converges quickly, a very large learning rate can sometimes lead to overshooting the minimum and oscillations, or even divergence.
3. **What happens if the learning rate is too small?**
   - If the learning rate is too small like **0.001** (as shown in the plot), the model converges very slowly. The loss decreases at a much more gradual pace, requiring significantly more epochs to reach a similar or optimal loss value.

It might also get stuck in local minima or take an impractical amount of time to train.

Exercise 2: Effect of Batch Size

Objective: To analyze how batch size influences training stability and speed.
Task: - Train the model using batch sizes: 1, 16, 64, and full dataset - Observe loss behavior

```
model_batch_1 = build_model()
model_batch_1.compile(optimizer=tf.keras.optimizers.SGD(learning_rate=0.01),
history_batch_1 = model_batch_1.fit(X, y, batch_size=1, epochs=100, verbose=

print(f"Training complete for batch size 1. Final loss: {history_batch_1.his
```

```
Training complete for batch size 1. Final loss: 0.0099
```

```
model_batch_16 = build_model()
model_batch_16.compile(optimizer=tf.keras.optimizers.SGD(learning_rate=0.01)
history_batch_16 = model_batch_16.fit(X, y, batch_size=16, epochs=100, verbo

print(f"Training complete for batch size 1. Final loss: {history_batch_16.hi
```

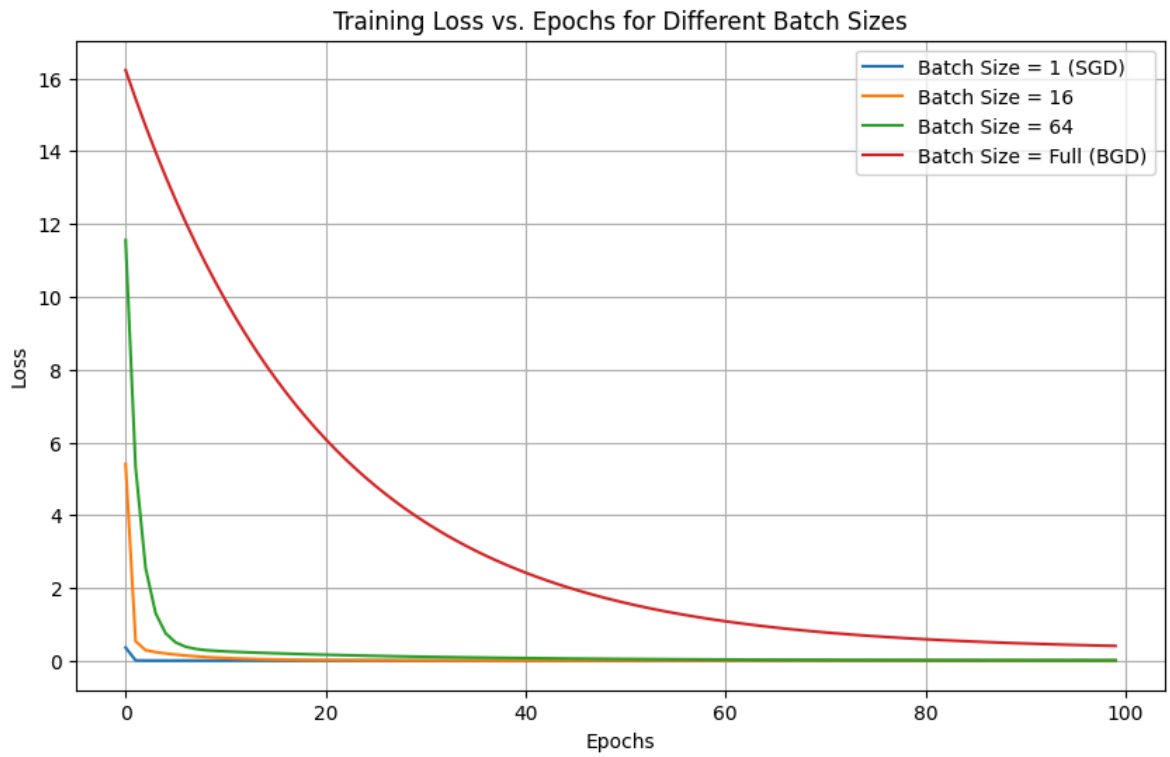
```
Training complete for batch size 1. Final loss: 0.0098
```

```
model_batch_64 = build_model()
model_batch_64.compile(optimizer=tf.keras.optimizers.SGD(learning_rate=0.01)
history_batch_64 = model_batch_64.fit(X, y, batch_size=64, epochs=100, verbo

print(f"Training complete for batch size 64. Final loss: {history_batch_16.h
```

```
Training complete for batch size 64. Final loss: 0.0098
```

```
model_batch_full = build_model()
model_batch_full.compile(optimizer=tf.keras.optimizers.SGD(learning_rate=0.0
history_batch_full = model_batch_full.fit(X, y, batch_size=len(X), epochs=10
```

```
plt.figure(figsize=(10, 6))
plt.plot(history_batch_1.history['loss'], label='Batch Size = 1 (SGD)')
plt.plot(history_batch_16.history['loss'], label='Batch Size = 16')
plt.plot(history_batch_64.history['loss'], label='Batch Size = 64')
plt.plot(history_batch_full.history['loss'], label='Batch Size = Full (BGD)'
plt.title('Training Loss vs. Epochs for Different Batch Sizes')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
plt.show()
```



Answers to Exercise 2 Questions

1. **Which batch size behaves like SGD?**
   - A batch size of **1** behaves like SGD, where parameters are updated after processing each single training example. This is evident from the highly fluctuating loss curve, as seen in the 'Batch Size = 1 SGD' plot, due to the noisy gradient estimates from individual samples.
2. **Which batch size behaves like Batch Gradient Descent?**
   - A batch size equal to the **full dataset** (in this case, `len(X)`) behaves like Batch Gradient Descent (BGD).
3. **Which batch size gives the best balance between speed and stability?**
   - **16 and 64** typically provide the best balance between speed and stability. They offer faster updates than BGD while providing more stable gradient estimates than SGD, leading to efficient convergence without excessive oscillations. The plots for batch sizes 16 and 64 show a good balance of

decreasing loss without the extreme fluctuations of SGD, or the slow, smooth descent of BGD.

```
Start coding or generate with AI.
```